

OnlyWheels

SYSTEM ARCHITECTURE

Online shop for wheels, tires and complete sets, with vehicle-fitment search, a 3D showroom, an AI photo studio and fitting at partner garages.

Strategy: web-first (PWA) → Android app from the same codebase via Capacitor.

Version 1.1 · Stack: React · Django · PostgreSQL · Redis · Celery · S3 · Capacitor

FRONTEND	BACKEND	DATABASE
React + Vite	Django + DRF	PostgreSQL

BACKGROUND JOBS	FILE STORAGE	MOBILE
Celery + Redis	S3 / R2	Capacitor (Android)

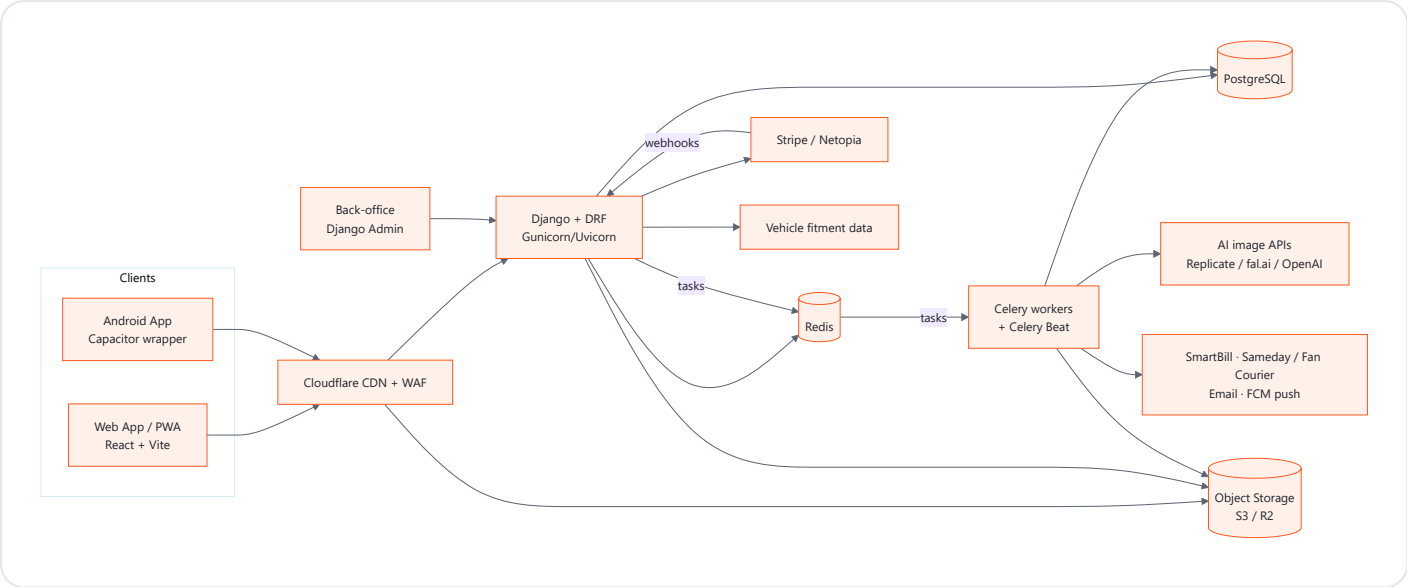
CONTENTS

01 Guiding principles — Core decisions behind the stack
02 High-level overview — How all components connect
03 Technology stack — Frontend, backend, AI, data, integrations
04 Repository structure (monorepo) — How the code is organised
05 Backend structure (Django apps) — Django apps and responsibilities
06 Data model (core entities) — Core entities and relations
07 Key flows — Fitment, 3D, AI studio, checkout
08 Requirements — Functional and non-functional
09 Actors and use cases — Who uses the system and how
10 Infrastructure & DevOps — Hosting, CI/CD, monitoring
11 Web → Android path — From PWA to the Play Store
12 Roadmap — Delivery phases

1. Guiding principles

Principle	Decision
Right tool per layer	**TypeScript** (JavaScript with types) **+ React** (UI library) on the frontend, **Python + Django** (web framework) on the backend
One codebase for web + Android	React **SPA** (single-page app – loads once, then updates without page reloads) / **PWA** (installable web app that can work offline), wrapped later with **Capacitor** (turns the web app into a native Android app)
Start simple, scale later	**Django monolith** (one backend application, deployed as a single unit) made of focused **Django apps** (self-contained modules, e.g. catalog, orders, payments)
Batteries included (the **framework** ships with most features built-in, so less code to write)	**Django ORM** (database access – work with tables as Python classes, no hand-written SQL), **migrations** (versioned, automatic database-structure changes), **auth** (user accounts, login, passwords, permissions) and **Django Admin** (ready-made admin web interface) as the **back-office** (internal panel for staff to manage products, orders, users)
Relational core data	**PostgreSQL** (relational database – data in linked tables) is the **source of truth** (the single authoritative place where data lives)
Async heavy work	**Celery** (background task runner) **+ Redis** (fast in-memory store, used as the task queue) for AI images, emails, invoices, courier **AWBs** (shipping labels / tracking numbers) – so users never wait for slow operations
AI as a service	Image generation through **external AI APIs** (paid online services called over the internet) – no **self-hosted models** or **GPUs** (no AI running on our own servers / graphics cards)
API contract first	**API** (the endpoints the app uses to talk to the backend) described by an **OpenAPI schema** (machine-readable description of every endpoint), generated by Django (**drf-spectacular**) → a **typed React client** (frontend code generated automatically from that schema, so frontend and backend always match)

2. High-level overview



Abbreviations

- **PWA** – Progressive Web App: a website that can be installed on the phone and work partly offline.
- **CDN** – Content Delivery Network: servers close to users that deliver images, 3D models and app files faster.
- **WAF** – Web Application Firewall: blocks attacks and bot traffic before it reaches the server.
- **DRF** – Django REST Framework: builds the REST API (the URLs the React app calls to get/send data).
- **S3 / R2** – Cloud file storage (Amazon S3 / Cloudflare R2) for photos, 3D models and invoices.
- **FCM** – Firebase Cloud Messaging: sends push notifications to phones and browsers.
- **API** – Application Programming Interface: the endpoints through which apps and services talk to each other.

---

3. Technology stack

3.1 Frontend – customer app (web + Android)

Concern	Technology
Framework	**React 19 + TypeScript**
Build tool	**Vite**
Routing	**TanStack Router** (type-safe) or React Router 7
Server state / data fetching	**TanStack Query** + client generated from Django's OpenAPI schema (**orval**)
Client state	**Zustand** (cart, configurator selections, UI)

Concern	Technology
Styling / UI kit	**Tailwind CSS** + **shadcn/ui** (Radix primitives)
Forms & validation	**React Hook Form** + **Zod** (Zod schemas can be generated from OpenAPI by orval)
3D showroom	**Three.js** via **react-three-fiber** + **@react-three/drei**; models in **glTF/GLB** (Draco + KTX2 compressed)
AR (later)	WebXR on web; **ARCore** via Capacitor plugin on Android
i18n	**i18next** (ro default, en)
PWA	**vite-plugin-pwa** (Workbox) – installable, offline catalog cache
Native wrapper	**Capacitor** (Android; iOS possible later) + plugins: Camera, Push Notifications, Share, Filesystem, Preferences
Analytics	**PostHog** (product analytics, feature flags)
Errors	**Sentry**

3.2 Back-office (admin)

Concern	Technology
Admin UI	**Django Admin** with a modern theme (**django-unfold**)
Features	Products & variants, 3D/image upload, stock, prices/promotions, orders, refunds, partner garages & slots, users, reviews moderation, AI-job monitoring & quotas
Import/export	**django-import-export** (bulk CSV/Excel product & stock import from suppliers)
History / audit	**django-simple-history** (who changed prices/stock/orders)

> A custom React admin is only needed later if partner garages get their own self-service portal.

3.3 Backend – API

Concern	Technology
Language / runtime	**Python 3.12**

Concern	Technology
Framework	**Django 5.x**
REST API	**Django REST Framework (DRF)**
OpenAPI / docs	**drf-spectacular** (Swagger UI + schema for the React client)
ORM / migrations	Django ORM + built-in migrations (`django.contrib.postgres` for JSONB, arrays, full-text)
Filtering / pagination	**django-filter** + DRF cursor pagination
Auth	**django-allauth** (email + Google/Apple login) + **djangorestframework-simplejwt** (access 15 min + rotating refresh token); **argon2** password hashing
Token storage	httpOnly secure cookie on web, `@capacitor/preferences` /secure storage on Android
Authorization	Groups/permissions: `customer` , `garage_partner` , `staff` , `admin` + DRF permission classes
Background jobs	**Celery** (workers) + **Celery Beat** (scheduled jobs) with **Redis** broker
Realtime updates	Polling for MVP → **Django Channels** (WebSocket) later for AI-job progress & order status
Caching / rate-limit	Redis via `django-redis` + DRF throttling
File storage	**django-storages** (S3 / Cloudflare R2) – pre-signed URLs for direct uploads
Images	**Pillow** + thumbnails via Cloudflare Images / imgproxy
Search	PostgreSQL full-text + `pg_trgm` → **Meilisearch** when the catalog grows
CORS / security	**django-cors-headers**, Django security middleware, CSP (`django-csp`)
Config	**django-environ** (12-factor `.env`)
Server	**Gunicorn** with Uvicorn workers (ASGI)
Dependency management	**uv** (fast, lockfile)
Logging	**structlog** → Sentry / Grafana Loki

3.4 AI photo studio (external APIs)

No models are run locally – Django calls hosted AI endpoints from Celery tasks.

Concern	Choice
Providers	**Replicate** or **fal.ai** (SDXL / Flux + ControlNet + inpainting) – main choice for precise wheel swaps; **OpenAI Images** / **Stability AI API** as alternatives
Inputs	3D-showroom render of the car with the chosen wheel, or the customer's own car photo + wheel mask
Abstraction	`ai_studio/providers/` – one small adapter per provider behind a common interface (`generate(scene, angle, style, image) → url`), switchable from settings
Execution	Celery task with retries + timeout; result stored in S3
Cost control	Per-user daily quota / credits, usage log per generation, provider cost tracked
Security	API keys only on the server (env vars), never in React/Android
Safety	Provider moderation + optional watermark for free tier

3.5 Data & storage

Store	Usage
PostgreSQL 16	Users, vehicles, fitment, products, stock, carts, orders, payments, appointments, reviews, AI generations
Redis 7	Cache, Celery broker/results, rate limits, (later) Channels layer
S3-compatible storage (Cloudflare R2 / AWS S3; **MinIO** locally)	Product images, 3D models (GLB), AI-generated photos, invoices (PDF)
Meilisearch (phase 4)	Fast faceted catalog search (brand, size, PCD, finish, price)

3.6 Third-party integrations (Romanian market)

Domain	Provider
Card payments	**Stripe** (Python SDK) and/or **Netopia Payments**
Installments (0% rate)	Netopia / bank partner (e.g. TBI Bank, card-based installments)

Domain	Provider
Invoicing + e-Factura (ANAF)	**SmartBill API** (or Oblio)
Shipping / AWB	**Sameday**, **Fan Courier** APIs
Vehicle fitment data	**Wheel-Size API** (PCD, center bore, ET, OEM sizes) – synced into PostgreSQL by Celery Beat
Email	**django-anymail** + Resend / SendGrid
Push notifications	**Firebase Cloud Messaging** via **fcm-django** (Android + web push)
AI images	Replicate / fal.ai / OpenAI / Stability AI
Maps (partner garages)	Google Maps / Mapbox

---

4. Repository structure (monorepo)

All code lives in one repository (**monorepo**) with two main parts – `frontend/` (React) and `backend/` (Django) – plus shared infrastructure and CI/CD configuration.

```
onlywheels/
├── frontend/                # React + Vite PWA (pnpm)
│   ├── src/
│   │   ├── api/             # generated client (orval) + TanStack Query hooks
│   │   ├── features/        # home, catalog, showroom3d, ai-studio, cart, account
│   │   ├── components/      # shared UI (shadcn/ui)
│   │   └── lib/             # native adapters (camera, share, push, storage)
│   └── android/             # generated by Capacitor (phase 3)
├── backend/                 # Django project (uv)
│   ├── config/
│   ├── apps/
│   ├── tests/
│   └── pyproject.toml
├── infra/
│   ├── docker-compose.yml   # postgres, redis, minio, mailpit, (meilisearch)
│   └── nginx/               # reverse proxy config (prod)
├── .github/workflows/       # CI/CD
└── README.md
```

---

5. Backend structure (Django apps)

Zooming into the `backend/` folder from the repository above – the Django project is split into focused apps:

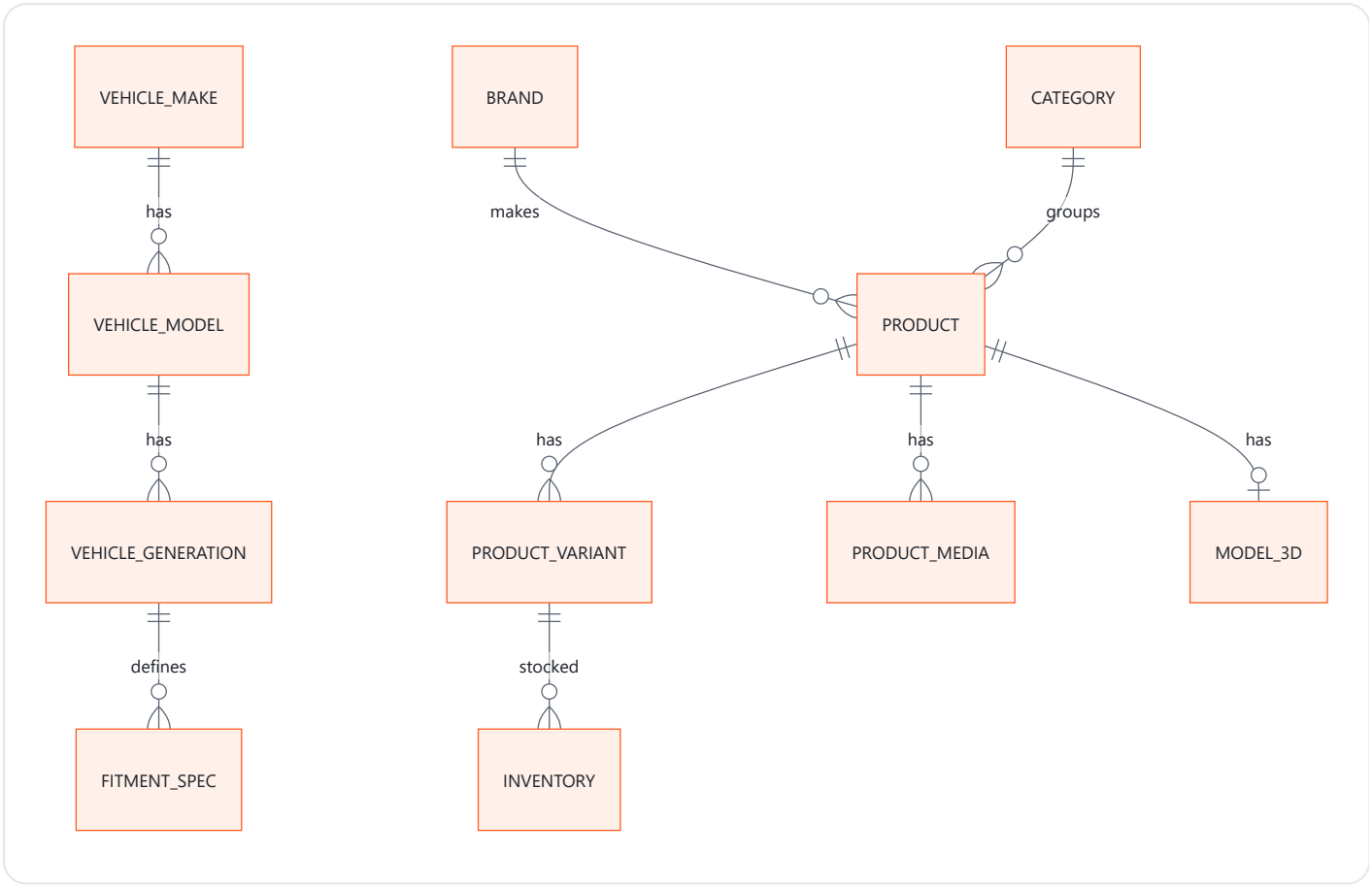
```
backend/
├── config/                 # settings (base/dev/prod), urls, asgi, wsgi, celery.py
└── apps/
    ├── accounts/           # custom User, profile, addresses, auth endpoints
    ├── vehicles/           # makes, models, generations, fitment specs, user's saved veh
    ├── catalog/            # brands, categories, products, variants, media, 3D models
    ├── fitment/            # "which wheels fit my car" compatibility engine
    ├── search/             # full-text search, (later) Meilisearch sync
    ├── pricing/            # promotions, vouchers
    ├── inventory/          # stock per variant, reservations
    ├── cart/               # cart, sets of 4, fitting add-on
    ├── orders/             # checkout, orders, status history, VAT
    ├── payments/           # Stripe / Netopia, webhooks, installments
    ├── shipping/           # AWB generation, tracking
    ├── invoicing/          # SmartBill, e-Factura
    ├── garages/            # partner garages, services, availability
    ├── appointments/       # fitting bookings
    ├── ai_studio/          # generations, gallery, quotas, providers/
    ├── reviews/            # ratings & reviews, favorites
    ├── notifications/      # email, push, in-app
    └── core/               # base models (UUID, timestamps), utils, permissions
```

Each app owns its models, `serializers.py`, `views.py` (DRF ViewSets), `services.py` (business logic), `tasks.py` (Celery) and `admin.py`. Apps communicate through services and Django signals, not through each other's internals.

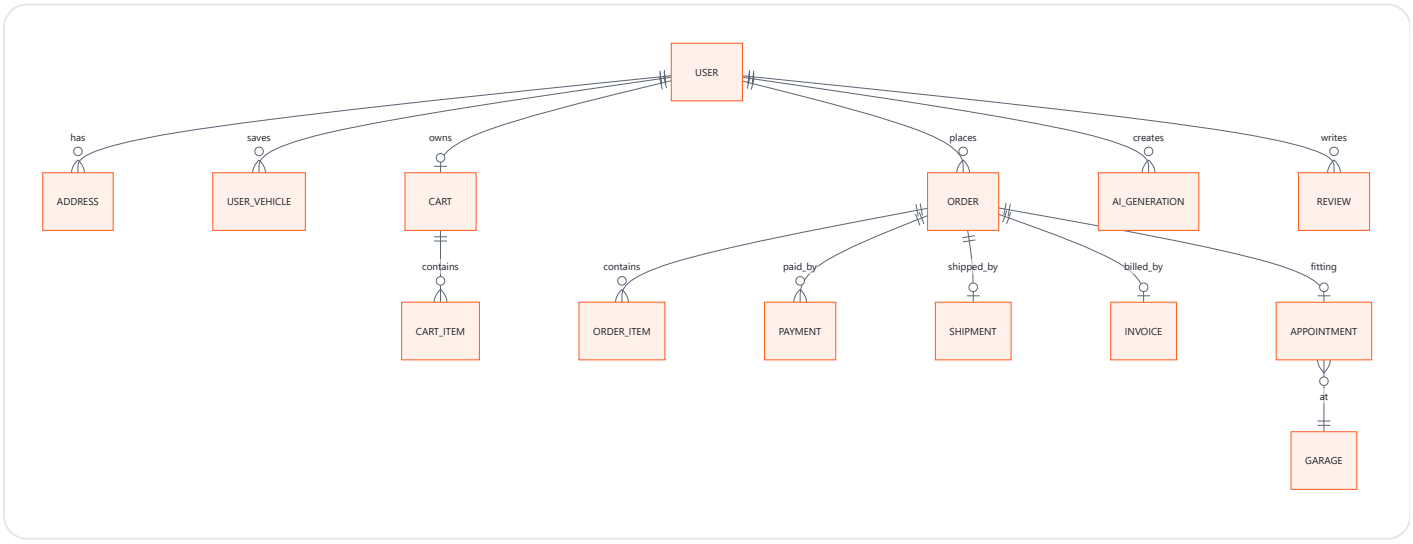
---

6. Data model (core entities)

Vehicles & catalog



Users, carts & orders



Cross-links: `USER_VEHICLE → VEHICLE_GENERATION`, `CART_ITEM / ORDER_ITEM → PRODUCT_VARIANT`, `REVIEW → PRODUCT`.

Key fields:

Model	Important fields
`FitmentSpec`	`generation, pcd (e.g. 5x114.3), center_bore_mm, et_min, et_max, oem_sizes (ArrayField), thread_size, max_load_kg`
`Product`	`type (wheel/tire/set/accessory), brand, category, name, slug, style, description, rating_avg, rating_count`
`ProductVariant`	`sku, diameter_in, width_in, et_mm, pcd, center_bore_mm, finish, price, old_price, currency`
`Model3D`	`glb_file, thumbnail, finish_materials (JSONField)`
`Order`	`number, status, subtotal, vat, shipping, total, payment_method, installments`
`Appointment`	`garage, slot_start, slot_end, status`
`AIGeneration`	`user, provider, source (3d_render/user_photo), scene, angle, style, status, result_file, cost`

Conventions: money as `DecimalField(max_digits=10, decimal_places=2)` in RON; `DateTimeField` with `USE_TZ=True`; primary keys as **UUID**; custom `User` model from day one.

---

7. Key flows

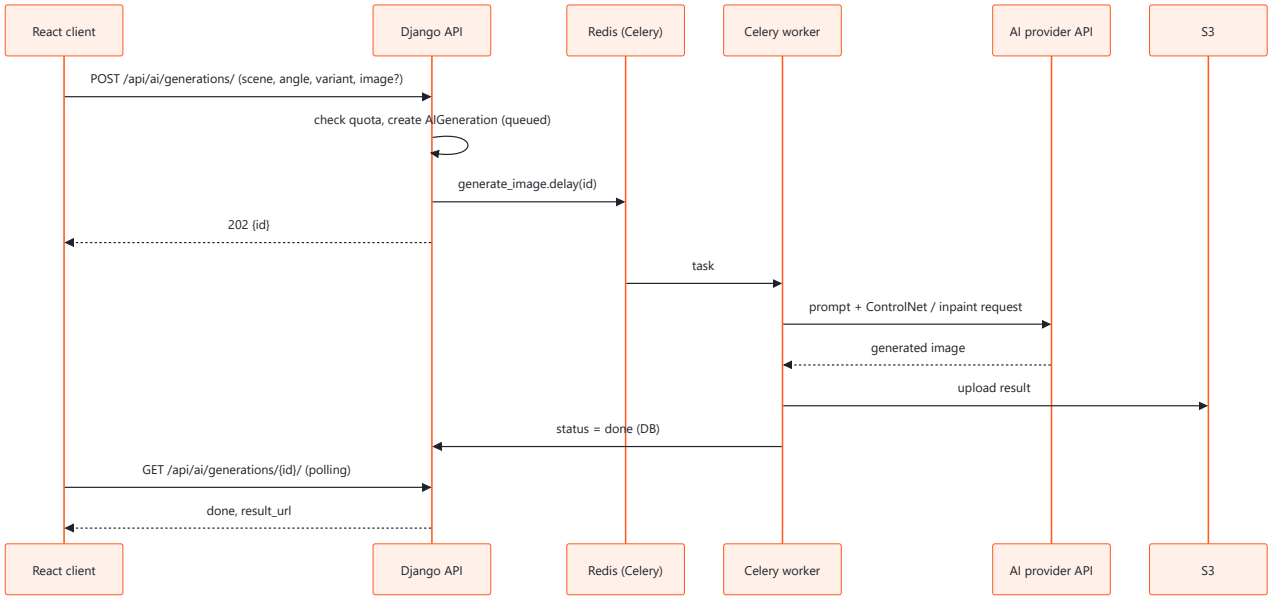
7.1 "Find wheels that fit my car"

1. User picks make → model → generation (or a saved vehicle).
2. `GET /api/fitment/{generation_id}/products/?…filters` checks matching **PCD**, compatible **center bore** (≤ hub, with hub rings), **ET** within range and allowed **diameters**.
3. Results are cached in Redis per generation + filter hash.

7.2 3D configurator

1. Frontend loads the car GLB plus the wheel GLB from the CDN (lazy, Draco-compressed).
2. Finish/size changes only swap materials and scale on the client, with no backend calls.
3. "Add to cart" sends the selected `variant_id` × 4.

7.3 AI photo studio



7.4 Checkout & payment

1. Cart → checkout (address, shipping or **fitting at partner garage** + slot).
2. Order is created as `pending_payment` and stock is **reserved** (15 min; expired by Celery Beat).
3. Redirect to Stripe/Netopia (card or installments).
4. Payment **webhook** → `paid` → Celery tasks: invoice (SmartBill), AWB (courier), confirmation email/push, garage notification.

---

8. Requirements

8.1 Functional requirements

ID	The user can…
FR1	Register / log in (email or Google) and save their car(s)
FR2	Search wheels **compatible with their car** (PCD, center bore, ET, size)
FR3	Browse and filter the catalog (brand, size, finish, price) and view product details
FR4	Preview wheels on the car in the **3D showroom** (change model, size, finish)
FR5	Generate **AI photos** of their car with the selected wheels
FR6	Add sets of 4 to the cart, pay online (card / installments) and track the order

ID	The user can…
FR7	Book **fitting at a partner garage** during checkout
FR8	Admin manages products, stock, prices, orders, garages and users
FR9	Partner garage sees and confirms its fitting appointments

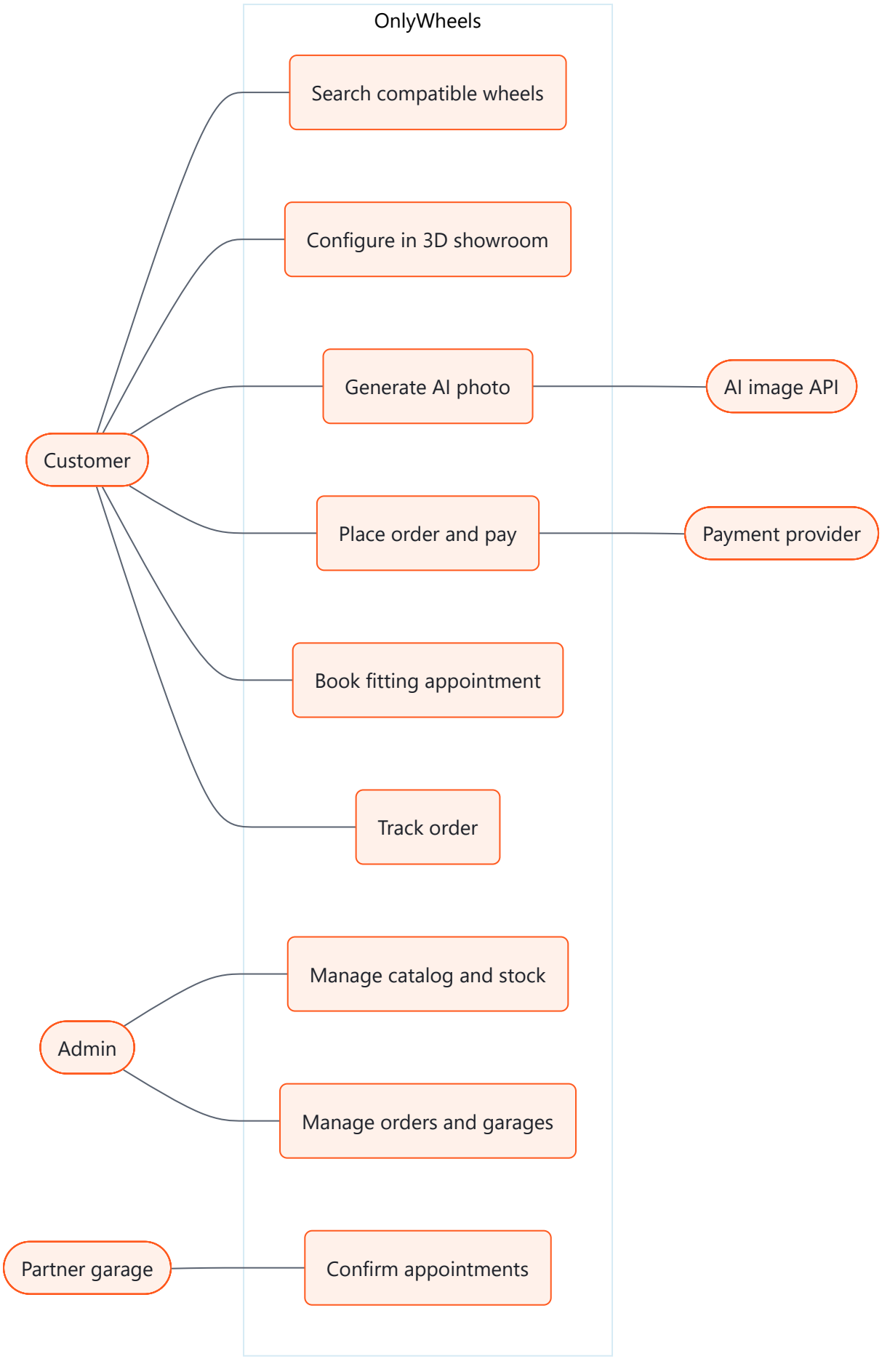
8.2 Non-functional requirements

Category	Requirement	Addressed by
Performance	Pages load in < 2 s on mobile; API responses < 300 ms	CDN, Redis cache, compressed 3D models
Responsiveness	Slow tasks (AI images, invoices) never block the user	Celery background jobs
Security	Encrypted traffic, safe passwords, protected payments	HTTPS, JWT, argon2, WAF, hosted payment page
Availability	≥ 99.5% uptime, daily backups	Managed PostgreSQL, monitoring (Sentry), backups
Scalability	Handle traffic peaks (e.g. tire-change season)	Stateless API, more workers/containers on demand
Portability	Same app on web and Android	React PWA + Capacitor
Usability	Mobile-first, Romanian & English	Responsive UI, i18next
Compliance	GDPR, Romanian e-Factura	EU data hosting, SmartBill integration

---

9. Actors and use cases

Actor	Description
Customer	Visitor or registered user who searches, configures and buys wheels
Admin	OnlyWheels staff managing catalog, orders and partners (via Django Admin)
Partner garage	Service that mounts the wheels and manages fitting appointments
External systems	Payment provider, AI image API, courier, invoicing (SmartBill)


OnlyWheels
Search compatible wheels
Configure in 3D showroom
Generate AI photo
AI image API
Customer
Place order and pay
Payment provider
Book fitting appointment
Track order
Manage catalog and stock
Admin
Manage orders and garages
Partner garage
Confirm appointments

10. Infrastructure & DevOps

Concern	Choice
Local dev	**Docker Compose**: PostgreSQL, Redis, MinIO, Mailpit; Django `runserver` + Vite dev server
Containers	Dockerfiles: `backend` (Django/Gunicorn), `worker` (Celery), `beat` (Celery Beat) – same image, different command
Hosting (MVP, low cost)	**Hetzner / DigitalOcean** VM with Docker + **Coolify**, or **Railway / Render**
Hosting (scale)	**AWS**: ECS Fargate (web/worker), RDS PostgreSQL, ElastiCache Redis, S3 + CloudFront
Frontend hosting	**Cloudflare Pages** / Vercel (static SPA + PWA)
Static files (admin)	**WhiteNoise** or S3
CDN / WAF / DNS	**Cloudflare**
CI/CD	**GitHub Actions**: ruff + mypy + pytest (backend), eslint + tsc + vitest (frontend) → build images → `migrate` → deploy; Android build with Gradle → Play Console (internal track)
Secrets	GitHub Environments + cloud secret manager; never in repo
Backups	Daily PostgreSQL dumps + PITR (managed DB), S3 versioning
Monitoring	**Sentry** (React + Django + Celery), **Flower** (Celery dashboard), Grafana/Loki or Better Stack, uptime checks

Environments: `local` → `staging` → `production`.

---

11. Web → Android path

1. Build a **responsive, mobile-first PWA** (bottom tab nav like the demo: Home, Catalog, Showroom 3D, AI Studio, Cart).
2. Keep native-dependent code behind small adapters (`camera` , `share` , `push` , `storage`) so the web version uses Web APIs and Android uses Capacitor plugins.
3. `npx cap add android` → configure deep links, splash/icons, FCM, ARCore plugin.
4. Use JWT storage via `@capacitor/preferences` / secure-storage instead of cookies on native; Django API is unchanged.
5. Publish via the Play Console. A **TWA** (Bubblewrap) is a lighter alternative if no native features are needed.

---

12. Roadmap

Phase	Scope
0 – Foundation	Repo, Docker Compose, CI, Django project + custom User, JWT auth, React app shell + design system
1 – MVP shop	Vehicle fitment search, catalog + filters, product page, cart (sets of 4), checkout with Stripe/Netopia, orders, Django Admin back-office, emails
2 – Experience	3D showroom configurator, AI studio (external APIs), reviews, favorites, partner garages + appointments, SmartBill + courier integration, PWA offline
3 – Android	Capacitor app, push notifications, camera upload for AI studio, Play Store release
4 – Scale	Meilisearch, Django Channels realtime, AR (ARCore/WebXR), installments, recommendations, garage partner portal, iOS